The Format Tax: What It Costs a Local LLM to Read, Understand and Write TXT, Markdown, LaTeX, HTML, JSON, PDF, DOCX, PPTX and XLSX

Luis Benvenuto[1]

[1]Independent Research

October 25, 2026

Abstract

People hand language models documents in many file formats and ask for files back in many more. We measure what each format costs a local model over the whole life of a document, with the content held fixed. We generate 18 synthetic reports with planted facts and render each one in twelve forms a model can be given: plain and marked-up text, text extracted from PDF, DOCX, PPTX and XLSX, and the raw XML inside the office files. We then ask three open models (1.5B–8B) about 15 questions per document and have them write the report back in thirteen deliverables, building and checking every file.

Reading is a token question. Extracted text costs what plain text costs. HTML and JSON cost 1.40 and 1.52 times as many tokens, and raw office XML 4.70–7.65 times. With the whole document in context, the format barely matters to Qwen3-8B (7-point spread; no format differs from Markdown after Holm correction). With the context cut to the plain text's length, raw XML falls to 15 % and JSON to 62 %, against 89 % for Markdown. Even among questions whose answer is still in the cut context, raw XML scores 71 % against 90 %. On a 12 GB consumer GPU, markup also pushes prompts past a memory cliff: above 1,440 tokens, prefill per token became up to 69 times slower.

Writing is a pipeline question, because a model never emits a PDF, DOCX or PPTX directly. We define the *format tax* as the time per valid file relative to Markdown, counting failures and retries. For Qwen3-8B, a PDF costs 1.0× through Markdown and 2.5× through LaTeX. A Word file costs 5.2× through python-docx. A slide deck costs 31.6×, and only 40 % of decks were ever valid. Two of the three models never produced a LaTeX file that compiled: they do not escape the dollar sign.

1 Introduction

Much everyday use of a language model is file work. People paste a PDF, upload a Word file or a slide deck, and ask for a report, a spreadsheet or a presentation back. Each step involves a format choice that is usually made by habit: extract the PDF or send its layout, keep the HTML tags or strip them, ask for LaTeX or for Markdown, have the model write python-docx code or the XML inside the .docx.

Earlier work measured slices of this. Prompt formatting changes accuracy [4, 12], table serialization changes table understanding [13], and forced output formats can hurt reasoning [14]. We know of no study that holds one document fixed and follows it through every common format, both into the model and out of it, and charges each format for everything it costs: tokens, wall-clock time, wrong answers, failed builds and retries.

That is what this paper does. We use local models, so every time is measured, not estimated, and nothing depends on an API's pricing. Our contributions:

- **One content, twelve read forms, thirteen deliverables.** A generator plants checkable facts in reports and renders each report, losslessly, into every format. The office formats can be read either as extracted text or as raw XML.
- **A markup cost model.** Tokens grow as a fixed cost plus a per-content multiplier. For raw slide XML the multiplier is 4.65: every word of content brings 4.65 tokens with it.
- **Markup dilution.** At the same token budget, markup costs accuracy beyond the content it pushes out of the window.
- **The memory cliff.** On a consumer GPU, markup does not only make reading proportionally slower. It can push the prompt past the point where the GPU runs out of memory and the driver spills to system RAM.
- **The format tax.** Time per valid file, relative to Markdown, for every route to a PDF, DOCX, PPTX

or spreadsheet, including the ones through code and raw XML.

2 Related work

Input format. Sui et al. [13] compare table serializations (Markdown, HTML, JSON, XML, ...) on structural understanding tasks and find that HTML with format explanations does well. He et al. [4] show that the same prompt written as plain text, Markdown, JSON or YAML can move GPT accuracy by tens of points. Sclar et al. [12] find similar sensitivity to formatting choices that carry no meaning. These studies vary the *prompt*. We vary the *document*: what the model is asked *about*, in the formats it comes in from files.

Output format. Tam et al. [14] show that forcing JSON or XML output can hurt reasoning. FOFO [15] measures whether models follow domain-specific formats. Both score the *text*. Neither builds the file nor asks what a valid DOCX, PDF, PPTX or XLSX costs, when the model can only produce it through code, markup or raw OOXML.

Document conversion and context. Pipelines turn PDFs and office files into model input with extractors and layout models (Docling [2]). The choice is usually made for extraction quality, not for what the result costs the model. Token cost differs by language under the same tokenizer [9], and models use long contexts unevenly [7]. We measure the analogous cost across formats, and a fixed-budget condition in which markup competes with content for the context.

3 Setup

Documents. A seeded generator writes "annual reports" of fictional networks. Each one has a title, author and date. Its sections hold paragraphs that each plant one fact (*In March, the Kadunnu observatory recorded 120 km of cable, according to a note filed by Orka Lolen*) plus filler sentences. There is also one bullet list, one table (unit, budget (k$), staff, score) and one equation. Every name is a random syllable string, so no answer comes from pretraining. About 15 questions per document cover fact values, people, table cells, the row with the most staff, list length and list items, which of two facts came earlier in the year, and the equation constant.

Formats. The same content tree is rendered into TXT, Markdown, JSON (indent 2), HTML5 and LaTeX by hand-written renderers, and into DOCX, PPTX and XLSX by python-docx, python-pptx and openpyxl. PDF is the LaTeX compiled by pdflatex. A model can receive a binary file in two ways: as the text a standard loader extracts (pypdf, python-docx, python-pptx, openpyxl) or as the raw XML parts of the zip package. That gives twelve *read formats*.

Models. Local, open-weight instruction models on one RTX 3060 (12 GB): Qwen2.5-1.5B-Instruct [10], Phi-3.5-mini-instruct [1], and Qwen3-8B [11] in NF4 4-bit [3] with thinking disabled. Decoding is greedy.

Read. We count tokens per format under five tokenizers. *Content density* is tokens(TXT)/tokens(format). Fitting tokens(format) $= a + b \cdot$tokens(TXT) over a size sweep (2, 4 and 8 sections) splits markup into a fixed cost a and a per-content cost b. Loader time is the median time to turn the file into the model's string.

Understand. The prompt places the document in `<document>` tags and appends the question. All questions about one (document, format) share a token prefix. It is prefilled once, timed with CUDA synchronization (the *read latency*), and each question is answered from a copy of its KV cache, as prefix caching does in serving systems [6]. A numeric answer is correct if the first number on the first answer line matches. A text answer is correct if the normalized gold appears in a short answer, and a comparison answer must not also name the other entity. In the *budget* condition each format is cut to the token count of its TXT version. Formats are compared with Markdown with exact McNemar tests [8] paired by question, Holm-adjusted [5].

Write. The model receives the report as plain text and one of ten instructions, which give the thirteen deliverables of Table 3. Each reply's first fenced block is built and validated:

- JSON must parse.
- LaTeX must compile with pdflatex.
- HTML and Markdown are turned into PDF with xhtml2pdf (via python-markdown for Markdown).
- python-docx, python-pptx and openpyxl code runs in a subprocess with a 60 s timeout and must save a file that opens.
- Raw `word/document.xml` is packed into a blank .docx that python-docx must open.
- CSV must have at least 2 rows of equal width.

A failed build is returned to the model with the error, up to 3 attempts. The cost of a deliverable is its *time per valid file*: all generation and build time, failures included, divided by the number of valid files. The **format tax** is that time divided by the same quantity for Markdown. Fidelity is the share of planted strings that the built

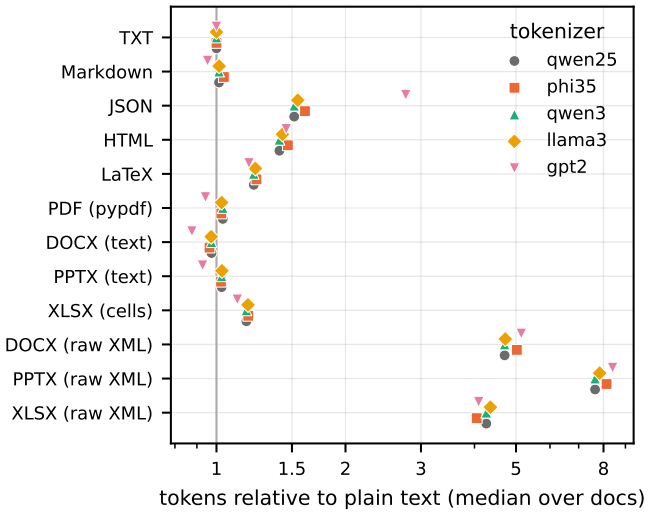


Figure 1: Tokens per format relative to the plain-text version of the same document (median over 18 documents), under five tokenizers.

Table 1: Read cost under the Qwen2.5 tokenizer. Density is tokens(TXT)/tokens(format). Fixed and per-TXT-token are the intercept and slope of tokens(format) = $a + b$·tokens(TXT) over the size sweep. Loader is the time to turn the file into a string.

Format	tokens/TXT	density	loader (ms)	fixed	per TXT tok
TXT	1.00	100%	–	-0	1.00
Markdown	1.01	99%	–	4	1.01
JSON	1.52	66%	–	73	1.41
HTML	1.40	71%	–	200	1.10
LaTeX	1.22	82%	–	127	1.03
PDF (pypdf)	1.03	97%	22.0	-24	1.07
DOCX (text)	0.97	103%	8.3	-18	1.00
PPTX (text)	1.03	97%	5.6	-12	1.05
XLSX (cells)	1.17	85%	3.0	64	1.08
DOCX (raw XML)	4.70	21%	0.2	2028	1.59
PPTX (raw XML)	7.65	13%	0.4	1946	4.65
XLSX (raw XML)	4.26	23%	0.2	1276	2.30

file, read back by the loaders, still contains. Spreadsheet routes write only the table and are scored on its cells.

4 Results

4.1 Read: tokens

Figure 1 and Table 1 give the read cost. The formats fall into three groups:

- **Extracted text costs what plain text costs.** The text that pypdf, python-docx, python-pptx and openpyxl extract is within a few percent of TXT. Markdown adds 1 %.
- **Markup costs 1.2–1.5×.** LaTeX costs 1.22×, HTML 1.40× and JSON 1.52×. The cost model separates them. HTML's overhead is mostly fixed (200 tokens of head and tags; slope 1.10). JSON's grows with content (slope 1.41): every string brings quotes, a key and indentation. Under GPT-2's tokenizer, which has no tokens for runs of spaces, JSON costs about 2.8×.
- **Raw office XML costs 4–8×.** DOCX XML costs 4.70×, XLSX XML 4.26× and PPTX XML 7.65× (7.6–8.4 across tokenizers). The slide XML's slope is 4.65: each word of content arrives wrapped in run, paragraph and shape elements.

Table 2: QA accuracy (%) with the full document and at the plain-text token budget, over 18 documents and about 15 questions each.

Format	phi35-mini		qwen25-1.5b		qwen3-8b	
	full	budget	full	budget	full	budget
TXT	93	93	79	79	91	91
Markdown	92	85	77	77	92	89
JSON	93	56	75	51	93	62
HTML	94	63	76	55	92	65
LaTeX	94	76	79	67	93	79
PDF (pypdf)	92	87	77	73	92	87
DOCX (text)	90	90	80	80	88	88
PPTX (text)	89	85	79	76	87	83
XLSX (cells)	90	74	76	66	90	75
DOCX (raw XML)	81	14	68	17	90	18
PPTX (raw XML)	76	8	61	8	89	9
XLSX (raw XML)	87	19	62	18	88	19

Loading is cheap next to reading: pypdf takes 22.0 ms per document, while the model's prefill takes about a second.

4.2 Understand: accuracy

With the whole document, format barely matters to the larger model. Qwen3-8B answers 92 % from Markdown and between 87 % and 93 % from every other format, raw XML included (Figure 2, Table 2). After Holm correction no format differs from Markdown (smallest adjusted $p = 0.073$). The smaller models do notice raw XML. Qwen2.5-1.5B drops from 77 % on Markdown to 61–68 % on the three XML forms, and Phi-3.5-mini from 92 % to 76–87 %, while it scores 89–94 % on every text and extracted form.

At a fixed budget, markup costs answers. When each format gets only as many tokens as the plain text, Markdown keeps 89 %. LaTeX falls to 79 %, HTML to 65 %, JSON to 62 %, and raw XML to 15 % on average. Most of that loss is content that no longer fits. But not all of it: among questions whose answer string is still inside the cut context, Qwen3-8B answers 90 % from TXT and Markdown and 71 % from raw XML. Markup dilutes the content it does not push out.

The memory cliff. Prefill time grows linearly with tokens, 1197 ms per thousand tokens for Qwen3-8B ($R^2 = 0.98$), until it doesn't (Figure 2, right). Above 1,440 prompt tokens for Qwen3-8B, 3,624 for Phi-3.5-mini and 7,206 for Qwen2.5-1.5B, the time per token

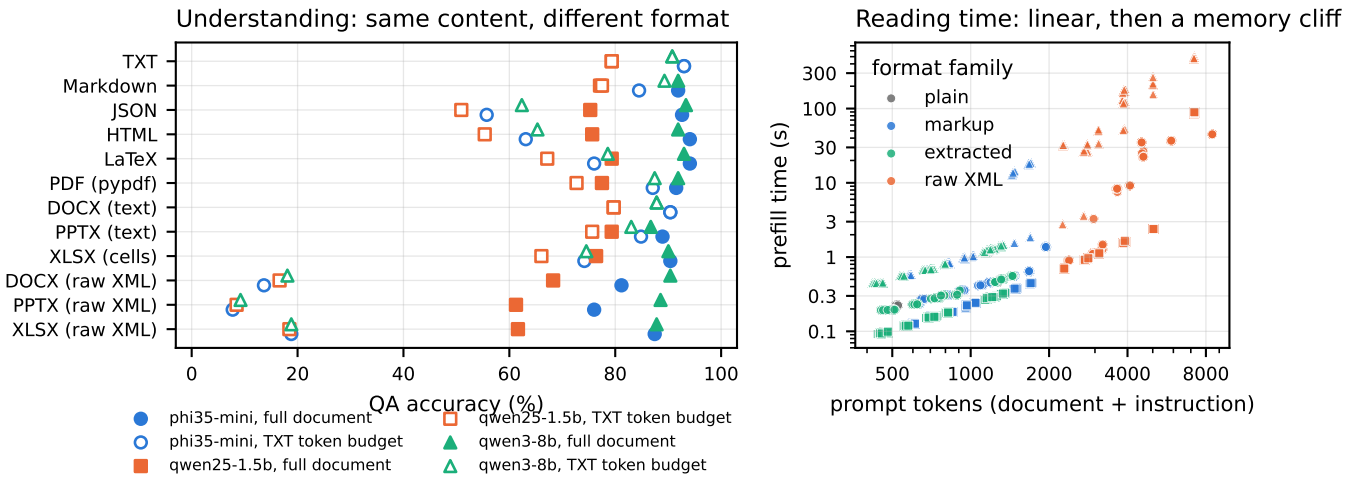


Figure 2: Left: QA accuracy per read format, with the full document (filled) and with the document cut to its plain-text token count (open). Right: prefill time against prompt length, one point per (document, format). Each family of points is linear until the prompt no longer fits in GPU memory, then rises steeply.

jumps, up to 69× the plain-text rate. The GPU has run out of memory, and the Windows driver spills to system RAM instead of failing. The one out-of-memory error we hit came from the attention call asking for 10.8 GiB at once, which suggests the attention matrix is materialized in full and grows with the square of the prompt. The prompts past the cliff are almost all raw XML and JSON. The median raw-PPTX prompt took 313.7× as long to read as its TXT version, where the token ratio alone predicts 7.65×. The cliff depends on the stack (GPU memory, attention kernel, driver), but the mechanism is general: markup moves a document toward whatever memory limit the hardware has.

4.3 Write: the format tax

Figure 3 and Table 3 give the cost of each deliverable. Text formats are cheap and safe: Markdown, TXT, JSON and HTML were valid for all three models, except one JSON file from the 1.5B model. Their cost follows their output tokens.

The PDF route decides the PDF's cost. The same PDF costs 1.0× Markdown through Markdown and xhtml2pdf, 1.7× through HTML and 2.5× through LaTeX for Qwen3-8B. LaTeX compiled on the first try for only 60 % of documents. Every failure had the same cause: the table header *Budget (k$)*, whose unescaped dollar sign opens math mode. Qwen3-8B fixed it on retry. Qwen2.5-1.5B and Phi-3.5-mini never did. They resent the same table every time, so their LaTeX route produced no PDF.

Word through code. A model writes a .docx only through python-docx code or raw XML. For Qwen3-8B, python-docx code worked on the first try for only 20 % of documents. It called methods that do not exist (`add_bullet_list`) or built a table with too few columns. Retries fixed every case, at a tax of 5.2×. Raw WordprocessingML was valid on the first try every time, at 4.4×. It costs 1.3× the tokens of the code but needs no repair. It also loses content: only 83 % of the planted facts survived, against 100 % for code. The 1.5B model produced no valid raw-XML document and a valid python-docx file for a third of the documents. Phi-3.5-mini's one raw-XML file that opened was blank: it put the text directly inside `<w:p>`, without the run and text elements Word displays, so the package is valid and the page is empty. It counts as valid in Table 3 and recovers 0 % of the facts.

Slides are the most expensive file. python-pptx decks were valid for 40 % of documents, at a tax of 31.6×, and Phi-3.5-mini produced none. Phi asked for placeholders the slide layout does not have (`slide.placeholders[1]`). On retry it moved to another missing index, or kept the same line and added the comment *Corrected line.* The 8B model's typical failure was a script that defines `create_presentation(output_path)` and never calls it. It was told, three times, that nothing had been saved. Its decks also carried only 36 % of the facts: a slide summarizes a paragraph instead of keeping it.

Spreadsheets are cheap because they are small. The XLSX and CSV routes write only the table, so their tax is below 1. Both models produced valid workbooks every time. The 1.5B model always saved them under a name of its own choosing instead of the path it was given, and its workbooks kept only about a third of the table cells.

Retries help only a model that can read the error. Of 48 retries by Qwen2.5-1.5B, 2 fixed the file, and 37

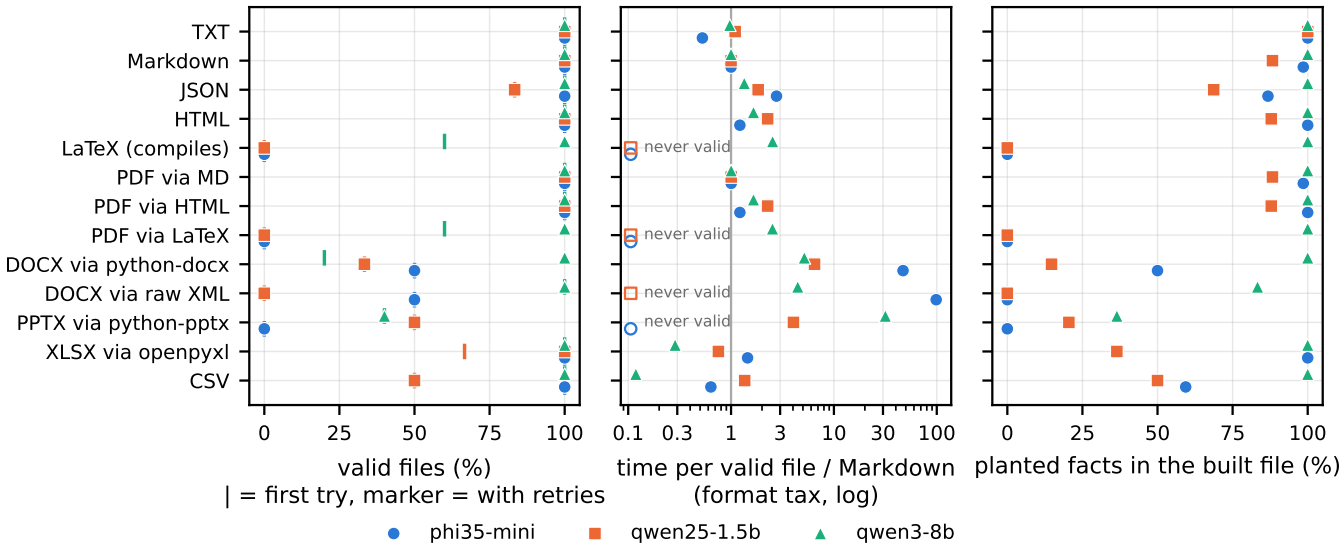


Figure 3: Writing each deliverable. Left: share of valid files, after retries (marker) and on the first try (bar). Middle: time per valid file relative to Markdown, the format tax (log scale). Right: planted facts that survive in the built file.

Table 3: Writing: valid files (%), median output tokens over all attempts, format tax (time per valid file / Markdown) and planted facts recovered from the built file (%). Up to 3 attempts, the build error fed back each time.

Deliverable	phi35-mini				qwen25-1.5b				qwen3-8b			
	valid	tokens	tax	facts	valid	tokens	tax	facts	valid	tokens	tax	facts
TXT	100	966	0.5	100	100	658	1.1	100	100	656	1.0	100
Markdown	100	1744	1.0	99	100	673	1.0	88	100	685	1.0	100
JSON	100	2874	2.8	87	83	888	1.8	69	100	895	1.3	100
HTML	100	2118	1.2	100	100	1010	2.3	88	100	1093	1.7	100
LaTeX (compiles)	0	6888	∞	0	0	2112	∞	0	100	757	2.5	100
PDF via MD	100	1744	1.0	99	100	673	1.0	88	100	685	1.0	100
PDF via HTML	100	2118	1.2	100	100	1010	2.3	88	100	1093	1.7	100
PDF via LaTeX	0	6888	∞	0	0	2112	∞	0	100	757	2.5	100
DOCX via python-docx	50	5376	47.0	50	33	1210	6.5	15	100	2156	5.2	100
DOCX via raw XML	50	7576	98.7	0	0	4016	∞	0	100	2803	4.4	83
PPTX via python-pptx	0	6102	∞	0	50	1250	4.0	20	40	5085	31.6	36
XLSX via openpyxl	100	618	1.4	100	100	384	0.8	36	100	178	0.3	100
CSV	100	306	0.6	59	50	315	1.4	50	100	73	0.1	100

failed again with the identical error. Qwen3-8B fixed 6 of 16, and Phi-3.5-mini 0 of 12. A retry loop multiplies the cost of a route by the number of attempts. It lowers the cost of a valid file only if the model can act on the error message.

4.4 Read and write together

Figure 4 puts the two costs on one plot. Reading and writing rank the formats differently. DOCX and PPTX are free to read once their text is extracted, and the most expensive files to write. JSON and HTML cost about 1.5× on both axes. Only TXT and Markdown cost about 1 on both axes, and only Markdown keeps the headings, list and table. A PDF built from Markdown is nearly as cheap.

5 Discussion

What to send a model. Extract the text. A PDF, DOCX, PPTX or XLSX read through a standard loader costs what plain text costs and answers as well. The loader takes milliseconds. Raw office XML costs 4–8× the tokens and, at any fixed budget, most of the answers. JSON and HTML are worth stripping when context is tight: they lose 20–30 points at the plain-text budget. With a large context and a capable model, format matters little to accuracy. It still matters to time, memory and money, which all scale with tokens, and on a local GPU possibly much faster than linearly.

What to ask a model for. Ask for Markdown, then convert. A PDF through Markdown cost as much as the Markdown itself. A PDF through LaTeX cost 2.5× for

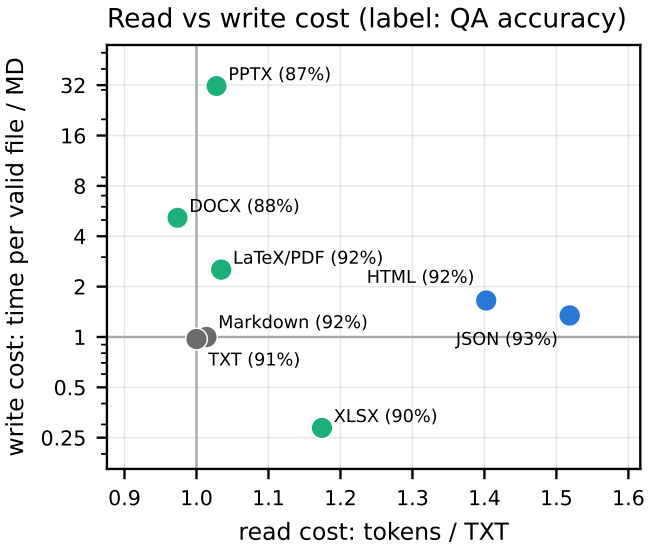


Figure 4: Each file type for Qwen3-8B: read cost (tokens of its extracted or marked-up text over TXT) against write cost (format tax). Labels give full-document QA accuracy.

the 8B model and was never produced by the two smaller models. LaTeX's failures were about escaping, not layout. Office files are the expensive end. The cost is not in the file but in the code that makes it, which the model has to get right against a library API, without seeing the result. A slide deck was the worst case on every measure. An agent that has to deliver a .docx or .pptx is better served by writing Markdown and converting it with a fixed, tested tool (pandoc, a template) than by writing python-docx or python-pptx code each time.

Raw XML is a surprise as an output. The 8B model never failed to produce a valid `document.xml`, while its python-docx code needed repair in most cases. For a capable model, the format with no API to misremember was the reliable one. It was not the faithful one, though: 83 % of the facts against 100 % for code.

Limitations.

- **Small, local models.** The largest model has 8B parameters, in 4-bit. Frontier models would fail less, and their tax would shrink towards the token ratio. Token counts do not depend on model quality. The write results are a lower bound on reliability.
- **Synthetic documents.** The reports are short (about 651 tokens of plain text) and regular. Real office files carry styles, revision marks and embedded objects that make raw XML heavier than ours, and longer documents would reach the memory cliff sooner.
- **Few documents for writing.** Writing used 6 documents for the 1.5B model, 5 for the 8B model (the sixth ran out of memory) and 2 for Phi-3.5-mini, which we stopped after 7 hours: past the memory cliff, one 8-section document would have taken most of a day. Validity rates are coarse at that size, and the tax ratios carry the variance of a few slow failures.
- **One stack.** The memory cliff belongs to one GPU, one attention implementation and one driver. Its position will move with all three. Its existence will not, as long as markup adds tokens.
- **Fidelity is string recall.** It checks that names and numbers survive, not layout, styling or whether a slide reads well.

6 Conclusion

The format a document arrives in, and the format a model is asked to produce, carry a cost that is easy to miss because it does not show up as an error. Reading, the cost is tokens. Markup and raw XML multiply them, dilute the content at a fixed budget, and on a consumer GPU can tip a prompt over a memory cliff. Writing, the cost is the pipeline. A PDF, Word file or slide deck is only ever as cheap as the text, code or XML the model can get right on the first try.

Markdown is the one format that is cheap to read and cheap to write, and it converts into a PDF at almost no extra cost. Binary office files are best read as extracted text. When they must be written, a model's code is the expensive part: a fixed converter is cheaper.

References

[1] Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Christoph Auer, Maksym Lysak, Ahmed Nassar, et al. Docling technical report. *arXiv preprint arXiv:2408.09869*, 2024.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X. Wang, and Sadid Hasan. Does prompt formatting have any impact on LLM performance? *arXiv preprint arXiv:2411.10541*, 2024.

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E.

Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[8] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[9] Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[12] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *Proc. ICLR*, 2024.

[13] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. Table meets LLM: Can large language models understand structured table data? a benchmark and empirical study. In *Proc. 17th ACM International Conference on Web Search and Data Mining (WSDM)*, 2024.

[14] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proc. EMNLP: Industry Track*, 2024.

[15] Congying Xia, Chen Xing, Jiangshu Du, Xinyi Yang, Yihao Feng, Ran Xu, Wenpeng Yin, and Caiming Xiong. FOFO: A benchmark to evaluate LLMs' format-following capability. In *Proc. ACL*, 2024.